

Short answer example:

If I have the following data: 5, 2, 6, 1, 4

How many operations are needed before I can confirm that it does not have a 3

(Ans: I need to check all 5, because the data is not sorted)

Code writing example: For the following singly linked code, add a function that counts the number of elements in the list and returns it:

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class LinkedList {
private:
    Node* head;

public:
    LinkedList() : head(nullptr) {}

    void insertAtHead(int val) {
        Node* newNode = new Node(val);
        newNode->next = head;
        head = newNode;
    }

    void deleteHead() {
        if (head == nullptr) {
            cout << "List is empty. Nothing to delete." << endl;
            return;
        }
        Node* temp = head;
        head = head->next;
        delete temp;
    }

    void display() {
        Node* current = head;
        while (current != nullptr) {
            cout << current->data << " -> ";
            current = current->next;
        }
        cout << "NULL" << endl;
    }

    ~LinkedList() {
        while (head != nullptr) {
            deleteHead();
        }
    }
};

int main() {
```

```

LinkedList list;

list.insertAtHead(10);
list.insertAtHead(20);
list.insertAtHead(30);

cout << "Current List: ";
list.display();

list.deleteHead();
cout << "After deleting head: ";
list.display();

return 0;
}

```

Determine the output examples.

```

class Node{
    public:
        int data;
        Node* next;
};
void main() {
    Node* head;
    head = new Node;
    head->data = 10;
    head->next = new Node;
    head->next->data = 20;
    head->next->next = 0;
    Node* boss = head;
    while (boss != 0) {
        cout << boss->data << " ";
        boss = boss->next;
    }
}

```

10 20

```

class Node{
    public:
        int data;
        Node* next;
};
void main() {
    Node* head;
    head = new Node;
    head->data = 10;
    head->next = new Node;
    head->next->data = 20;
    head->next->next = new Node;
    head->next->next->data = 5;
    head->next->next->next = 0;
    Node* boss = head;
    int temp = boss->data;
    while (boss != 0) {
        if (boss->data > temp)
            temp = boss->data;
        boss = boss->next;
    }
    cout << temp;
}

```

20

```

}
void main() {
    int* intptr;
    intptr = new int[20];
    for (int i = 0; i < 20; i++)
        intptr[i] = i*2;
    int* newptr = intptr;
    newptr = newptr + 4;
    cout << *newptr;
}
class List{
    public:
        int* data;
        List(int num);
};
List::List(int num) {
    data = new int[10];
    for (int i = 0; i < 10; i++)
        data[i] = i * num;
}
void main() {
    List list1(2);
    cout << list1.data[3];
}
class List{
    public:
        int* data;
        List(int num);
};
List::List(int num) {
    data = new int[10];
    for (int i = 0; i < 10; i++)
        data[i] = i * num;
}
void main() {
    List list1(2); List list2(list1); list2.data[6] = 100;
    cout << list1.data[6];
}
class List {
    public:
        int* data;
        List(int num);
};
List::List(int num) {
    data = new int[10];
    for (int i = 0; i < 10; i++)
        data[i] = i * num;
}
void main() {
    List list1(2); List list2(1);
    cout << list1.data[4] + list2.data[2];
}
class List{
    public:
        int* data;
        List(int num);
};
List::List(int num) {

```

8

6

100

10

12

```

        data = new int[10];
        for (int i = 0; i < 10; i++)
            data[i] = i * num;
    }
    void main() {
        List list1(2); List list2(1);
        list2.data[6] = 100;
        cout << list1.data[6];
    }
    class List{
        public:
            int* data;
            List(int num);
            List(List& other_list);
    };
    List::List(int num) {
        data = new int[10];
        for (int i = 0; i < 10; i++)
            data[i] = i * num;
    }
    List::List(List& other_list) {
        data = new int[10];
        for (int i = 0; i < 10; i++)
            data[i] = other_list.data[i];
    }
    void main() {
        List list1(2); List list2(list1); list2.data[6] = 100;
        cout << list1.data[6] << " ";
        cout << list2.data[6];
    }
    class List {
        public:
            int* data;
            List(int num);
    };
    List::List(int num) {
        data = new int[10];
        for (int i = 0; i < 10; i++)
            data[i] = i * num;
    }
    void main() {
        List list1(3); List list2 = list1;
        list1.data[6] = 100;
        cout << list2.data[6];
    }

```

12 100

100

Draw structure and determine output example.

```

class Node{
    public:
        int data;
        Node* next;
        Node* prev;
};

void main() {

```

```

Node *n1, *n10, *n20, *n30, *n100, *n200, *n300;
n1 = new Node;
n10 = new Node; n20 = new Node; n30 = new Node;
n100 = new Node; n200 = new Node; n300 = new Node;

n1->data = 1; n1->next = n100; n1->prev = n30;
n10->data = 10; n10->next = n20; n10->prev = n1;
n20->data = 20; n20->next = n30; n20->prev = n10;
n30->data = 30; n30->next = n1; n30->prev = n20;
n100->data = 100; n100->next = n200; n100->prev = n1;
n200->data = 200; n200->next = n300; n200->prev = n100;
n300->data = 300; n300->next = n1; n300->prev = n200;

Node* cursor = n1->next;
while (cursor != n1) {
    cout << cursor->data << " ";
    cursor = cursor->next;
}
cout << cursor->data << " ";
cursor = cursor->prev;
while (cursor != n1) {
    cout << cursor->data << " ";
    cursor = cursor->prev;
}
cout << cursor->data << " ";
}

```

Output:
100 200 300 1 30 20 10 1

